JAVA 17

Masterclass Notes

Core Java • Modern Language Features • Collections • Streams • Concurrency • JVM • DSA • FAANG-Style Interview Questions

Designed for	Beginners progressing toward backend engineering and technical interviews
Includes	Concept explanations, runnable examples, production considerations, complexity notes, common traps, and Q&A
Version focus	Java 17 LTS; clearly distinguishes Java 17 features from features introduced in earlier releases

How to use these notes

• Read the concept, run the example, then explain it without looking at the page.

• For each topic, practise one implementation and one interview question.

• For coding interviews, always state assumptions, edge cases, time complexity, and space complexity.

• Treat examples as teaching snippets; production systems also need tests, observability, security, and error handling.

Prepared as a study guide. Always verify version-specific behavior in the official Java 17 API and language documentation.

Contents

- 1. Java platform and execution model
- 2. Core language foundations
- 3. Object-oriented programming
- 4. Strings, equality, and immutability
- 5. Exceptions and resource management
- 6. Collections and generics
- 7. Lambdas and functional interfaces
- 8. Stream API and Optional
- 9. Java 17 language features
- 10. Date and time API
- 11. Concurrency and the Java Memory Model
- 12. JVM, memory, and garbage collection
- 13. I/O, NIO, modules, and useful APIs
- 14. Clean code, testing, and production guidance
- 15. DSA patterns and complexity
- 16. Interview questions with answers
- 17. Final revision checklist and study plan

Version accuracy note

Java 17 is an LTS release from September 2021. It includes language, library, JVM, security, and platform changes from earlier releases. A feature available while running Java 17 is not necessarily a feature introduced in Java 17. Preview features are not finalized and may require explicit compiler/runtime flags.

1. Java platform and execution model

Java source code is compiled by `javac` into bytecode (.class files). A compatible Java Virtual Machine (JVM) loads, verifies, interprets, and may JIT-compile bytecode into native machine code.

```
javac Main.java
java Main
```

JDK, JVM, and runtime

• JDK: development kit containing tools such as javac and supporting runtime components.

• JVM: virtual machine that executes Java bytecode.

• JRE: traditional term for a runtime environment containing the JVM and libraries; packaging differs across modern distributions.

Why Java is platform-independent

The compiler targets JVM bytecode rather than one operating system's machine code. A compatible JVM can execute that bytecode on different platforms. Platform independence does not mean every program behaves identically in every environment: native libraries, file paths, encodings, permissions, and runtime versions can still matter.

A minimal Java program

```
public class Main {
    public static void main(String[] args) {
        System.out.println("Hello, Java 17!");
    }
}
```

main is the conventional entry point for a simple Java application. `String[] args` receives command-line arguments.

Compilation versus execution

• Compile-time errors include type mismatches and invalid syntax.

• Runtime exceptions occur while the program executes.

• The JIT compiler optimizes frequently executed code; Java is not simply interpreted line by line.

• Bytecode verification helps enforce structural and type-safety constraints.

2. Core language foundations

Primitive and reference types

Category	Types / examples	Key point
Integral primitives	byte, short, int, long	Whole-number types; long literals may use L.
Floating point	float, double	Binary floating point; not ideal for exact decimal money.
Other primitives	char, boolean	char is a UTF-16 code unit; boolean is true/false.
Reference types	String, arrays, classes, interfaces, records	Variables hold references; references can be null.

Variables and type inference

```
int count = 3;
long population = 8_000_000_000L;
double ratio = 0.75;
var message = "type inferred as String";
```

`var` is local-variable type inference, introduced in Java 10. The type is determined at compile time; Java does not become dynamically typed. It cannot be used for fields, method parameters, or return types.

Operators and control flow

- Arithmetic: +, -, *, /, %. Integer division truncates toward zero.
- Comparison: ==, !=, <, <=, >, >=. For reference values, == compares identity, not logical equality.
- Logical: && and || short-circuit; & and | can also operate on booleans without short-circuiting.
- Control flow: if/else, switch, for, enhanced for, while, do/while, break, continue, return.
- Use braces consistently, meaningful names, and guard clauses to reduce nesting.

Arrays

```
int[] values = {4, 8, 15};
System.out.println(values.length); // 3

for (int value : values) {
    System.out.println(value);
}
```

Arrays have fixed length. Indexed access is O(1); searching an unsorted array is O(n). Java checks array bounds at runtime.

Methods and parameter passing

Java is always pass-by-value. For object arguments, the copied value is a reference, so a method can mutate the referenced object but cannot replace the caller's variable by reassigning its local parameter.

```
static void rename(StringBuilder value) {
    value.append("!");
}

StringBuilder text = new StringBuilder("Java");
rename(text);
System.out.println(text); // Java!
```

Numeric pitfalls

- Integer overflow wraps around for ordinary integer arithmetic; it does not automatically throw.
- Use long when the possible range exceeds int.
- Use BigDecimal with a deliberate rounding policy for exact decimal business calculations.
- Beware of floating-point comparison: 0.1 + 0.2 is not exactly 0.3 in binary floating point.

3. Object-oriented programming (OOP)

The four pillars

Pillar	Meaning	Example
Encapsulation	Protect state behind a clear API.	Private account balance; deposit/withdraw methods.
Abstraction	Expose essential behavior, hide implementation details.	PaymentProcessor interface.
Inheritance	Reuse/extend behavior through an is-a relationship.	CardPayment extends PaymentMethod.
Polymorphism	Same interface, different runtime behavior.	processor.process(payment).

Class, object, constructor

```
public class BankAccount {
    private final String accountId;
    private long balanceInCents;

    public BankAccount(String accountId) {
        if (accountId == null || accountId.isBlank()) {
            throw new IllegalArgumentException("accountId required");
        }
        this.accountId = accountId;
    }

    public void deposit(long cents) {
        if (cents <= 0) throw new IllegalArgumentException("positive amount required");
        balanceInCents = Math.addExact(balanceInCents, cents);
    }

    public long getBalanceInCents() {
        return balanceInCents;
    }
}
```

The example uses integer cents to avoid floating-point representation problems. Real financial systems also need currency handling, authorization, persistence, transaction boundaries, and auditability.

Overloading versus overriding

• Overloading: same method name, different parameter list; resolved at compile time.

• Overriding: subclass provides an implementation for an inherited instance method; selected by runtime dispatch.

• A static method is hidden, not overridden in the polymorphic instance-method sense.

• Use @Override to catch signature mistakes.

Abstract classes versus interfaces

• An abstract class can hold instance state and constructors, and can define abstract and concrete methods.

• An interface defines a contract and can have abstract, default, static, and private methods.

• A class can implement multiple interfaces but directly extend only one class.

• Prefer composition over inheritance when a relationship is not a genuine, stable is-a relationship.

Access modifiers

Modifier	Visibility
private	Within the declaring top-level class's permitted nested context.
package-private (no modifier)	Within the same package.
protected	Same package, plus subclass access subject to Java's rules.
public	Accessible wherever the enclosing type/module rules permit.

static and final

• static members belong to the class rather than an individual instance.

• final variable: assign once; for a reference, the reference cannot be reassigned, but the referenced object may still be mutable.

• final method cannot be overridden; final class cannot be subclassed.

• static initializers run during class initialization; avoid complicated side effects there.

4. Strings, equality, and immutability

String is immutable

A String's contents cannot be changed after creation. Operations such as replace or toUpperCase return a String result; they do not mutate the original.

```
String name = "java";
String upper = name.toUpperCase();
System.out.println(name);  // java
System.out.println(upper); // JAVA
```

== versus equals

```
String a = new String("java");
String b = new String("java");

System.out.println(a == b);      // false: different objects
System.out.println(a.equals(b)); // true: same character content
```

Use `equals` for logical equality. String literals may be interned, so comparing literals with == can appear to work; do not rely on that for content equality.

equals and hashCode contract

• If a.equals(b) is true, a.hashCode() must equal b.hashCode().

• Equal hash codes do not imply objects are equal.

• Equality should be reflexive, symmetric, transitive, consistent, and false for null.

• When overriding equals, normally override hashCode too.

• Do not mutate fields used in equality/hash calculation while an object is being used as a HashMap key or HashSet element.

StringBuilder and StringBuffer

Type	Characteristics	Use
String	Immutable	Values that should not be modified in place.
StringBuilder	Mutable, not synchronized	Efficient string construction within one thread.
StringBuffer	Mutable, synchronized methods	Legacy cases requiring its synchronization behavior.

Useful Java 11 String methods

```
String input = "  Java 17  ";
input.strip();             // "Java 17"
"   ".isBlank();           // true
"ab".repeat(3);            // "ababab"
"a\\nb".lines().count();  // 2
```

`strip()` uses Unicode-aware whitespace handling; `trim()` follows older, narrower character rules.

5. Exceptions and resource management

Exception hierarchy

• Throwable is the root of Java's throwable types.

• Error generally represents serious conditions applications do not normally try to recover from.

• Exception includes conditions applications may handle.

• Checked exceptions must be caught or declared, subject to Java's rules.

• RuntimeException and its subclasses are unchecked.

try/catch/finally and try-with-resources

```
import java.io.BufferedReader;
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;

class ReadFile {
    static String firstLine(Path path) throws IOException {
        try (BufferedReader reader = Files.newBufferedReader(path)) {
            return reader.readLine();
        }
    }
}
```

Try-with-resources closes AutoCloseable resources automatically, including when an exception occurs. Prefer it for files, streams, sockets, and similar resources.

Good exception practices

- Throw specific exceptions and provide useful context.
- Do not swallow exceptions silently.
- Do not use exceptions for ordinary expected branching when a simpler result type is appropriate.
- Preserve the original cause when wrapping an exception.
- Avoid logging the same exception at every layer; define clear ownership of logging and error translation.

Custom exception

```
class InsufficientFundsException extends RuntimeException {
    public InsufficientFundsException(String message) {
        super(message);
    }
}
```

6. Collections and generics

Collection framework map

List preserves sequence and allows duplicates. Set represents uniqueness. Queue/Deque model processing order. Map stores key-value mappings and is not a subtype of Collection.

Type	Typical complexity	Notes
ArrayList	get O(1); middle insert/remove O(n)	Dynamic array; good indexed reads.
LinkedList	search O(n); endpoint operations O(1)	Doubly linked list; poor random access.
HashSet	average add/contains/remove O(1)	No guaranteed iteration order.
TreeSet	O(log n)	Sorted unique elements.
HashMap	average get/put O(1)	No guaranteed iteration order; permits one null key.
TreeMap	O(log n)	Sorted keys; natural ordering or comparator.
ArrayDeque	amortized O(1) at ends	Usually preferred over Stack; no null elements.
PriorityQueue	peek O(1), offer/poll O(log n)	Head is the least element by default.

HashMap at a high level

- The key's hash helps select a bucket; equality distinguishes keys within the bucket.
- Collisions are expected and handled by the implementation.
- Resizing occurs as the map grows beyond its capacity/threshold policy.

• Modern implementations can use tree bins for heavily collided buckets under specified conditions.

• Expected constant-time behavior assumes suitable hash distribution; it is not an unconditional guarantee.

• HashMap is not thread-safe. Use ConcurrentHashMap or explicit synchronization when shared concurrent mutation is required.

Counting with Map.merge

```
import java.util.HashMap;
import java.util.Map;

String[] words = {"java", "sql", "java"};
Map<String, Integer> counts = new HashMap<>();

for (String word : words) {
    counts.merge(word, 1, Integer::sum);
}
// java -> 2, sql -> 1
```

Comparable and Comparator

```
import java.util.Comparator;
import java.util.List;

record Employee(String name, int salary) {}

List<Employee> employees = List.of(
    new Employee("Mina", 90000),
    new Employee("Dev", 80000)
);

employees.stream()
    .sorted(Comparator.comparingInt(Employee::salary).reversed())
    .forEach(System.out::println);
```

Comparable defines a type's natural ordering. Comparator defines an external or alternative ordering. Ensure ordering is consistent with equality where required by sorted sets/maps.

Generics and type erasure

```
static <T> T first(List<T> items) {
    if (items.isEmpty()) throw new IllegalArgumentException("empty list");
    return items.get(0);
}
```

• Generics provide compile-time type safety and reduce casts.

• Java implements generics primarily through type erasure.

• You cannot use primitive types directly as type arguments: use Integer, Long, and so on.

• PECS: Producer Extends, Consumer Super — a useful guideline for wildcard APIs.

• List is useful for reading Number values; List can accept Integer values.

Immutable factory collections

```
List<String> names = List.of("Ava", "Noah");
Set<Integer> ids = Set.of(1, 2, 3);
Map<String, Integer> scores = Map.of("Ava", 90, "Noah", 85);
```

These factory methods create unmodifiable collections and reject null elements/keys/values. They are not deep immutability guarantees if elements themselves are mutable.

7. Lambdas and functional interfaces

A functional interface has one abstract method. Its instances can be represented by lambdas or method references. Default and static methods do not count as abstract methods.

Interface	Main method	Typical use
Predicate	boolean test(T)	Filter or validate.

Interface	Main method	Typical use
Function	R apply(T)	Transform a value.
Consumer	void accept(T)	Perform an action.
Supplier	T get()	Create/provide a value.
UnaryOperator	T apply(T)	Transform T to T.
BinaryOperator	T apply(T,T)	Combine two values of same type.

```
import java.util.function.Predicate;

Predicate<Integer> isEven = n -> n % 2 == 0;
System.out.println(isEven.test(8)); // true
```

A lambda's `this` refers to the enclosing context; an anonymous class has its own instance context. Captured local variables must be final or effectively final.

Method references

- Static method: Integer::parseInt
- Instance method of a particular object: System.out::println
- Instance method of an arbitrary object of a type: String::toUpperCase
- Constructor reference: ArrayList::new

8. Stream API and Optional

Stream pipeline

A stream does not store the data source. Intermediate operations such as filter/map are lazy; a terminal operation triggers processing. A stream should not be reused after a terminal operation.

```
import java.util.List;

record Order(long id, String status, long amountInCents) {}

List<Order> orders = List.of(
    new Order(1, "PAID", 10000),
    new Order(2, "PENDING", 5000),
    new Order(3, "PAID", 25000)
);

long revenue = orders.stream()
    .filter(o -> o.status().equals("PAID"))
    .mapToLong(Order::amountInCents)
    .sum();
// 35000 cents
```

Important operations

Operation	Meaning / caution
filter	Keep elements satisfying a predicate.
map	Transform each element one-to-one.
flatMap	Transform each element to a stream and flatten.
distinct	Remove duplicates according to equality.
sorted	Sort; can require buffering.
collect	Accumulate into a collection or summary.
groupingBy	Group values by classifier.
reduce	Combine elements using an associative operation where needed.
findFirst / findAny	Find an element; parallel behavior differs.
anyMatch / allMatch / noneMatch	Short-circuiting boolean tests.

Grouping example

```
Map<String, List<Order>> byStatus = orders.stream()
    .collect(java.util.stream.Collectors.groupingBy(Order::status));
```

Stream best practices

• Avoid mutating shared external state inside stream operations.

• Do not assume parallel streams are faster; benchmark representative workloads.

• Be careful with ordering requirements and side effects.

• For monetary values, use BigDecimal or integer minor units rather than double where precision matters.

Optional

```
Optional<String> email = Optional.of("dev@example.com");

String safe = email
    .filter(s -> s.contains("@"))
    .orElse("unknown@example.com");
```

• of rejects null; ofNullable maps null to empty; empty creates an empty Optional.

• orElse evaluates its argument eagerly; orElseGet evaluates a supplier only when empty.

• map transforms a present value; flatMap is useful when the mapper itself returns Optional.

• Optional is often useful as a return type for possible absence, not as a mandatory wrapper around every field or parameter.

9. Java 17 language features

Feature timeline

Feature	Release status	What to remember
Modules / List.of	Java 9	Modularization and unmodifiable collection factories.
String.isBlank, strip, lines, repeat	Java 11	Modern string utilities.
var for local variables	Java 10	Compile-time type inference only.
Switch expressions	Final in Java 14	Switch can return a value; use yield for block results.
Text blocks	Final in Java 15	Readable multiline string literals.
Records	Final in Java 16	Concise data-carrier types; shallow immutability only.
Pattern matching instanceof	Final in Java 16	Type test and binding in one expression.
Sealed classes/interfaces	Final in Java 17	Restrict direct permitted subtypes.
Pattern matching for switch	Preview in Java 17	Not finalized in Java 17; avoid treating as standard finalized syntax.
Foreign Function & Memory API	Incubator in Java 17	Experimental API status; not a stable core interview feature.

Records

```
public record Customer(long id, String name) {
    public Customer {
        if (id <= 0) throw new IllegalArgumentException("id must be positive");
        if (name == null || name.isBlank()) throw new IllegalArgumentException("name required");
    }
}

Customer c = new Customer(101, "Aarav");
System.out.println(c.id()); // record accessor, not getId()
```

- Records implicitly extend java.lang.Record and cannot extend another class.
- They can implement interfaces and define additional methods and static members.
- Component fields are final, but referenced mutable objects may still change.
- Use records for value-like data carriers and DTOs; do not force every domain entity into a record.

Sealed classes and interfaces

```
public sealed interface Payment
        permits CardPayment, BankPayment {}

public final class CardPayment implements Payment {}
public final class BankPayment implements Payment {}
```

Each permitted direct subtype must be final, sealed, or non-sealed. Sealed types help represent a closed set of domain variants.

Pattern matching for instanceof

```
static String describe(Object value) {
    if (value instanceof String text) {
        return "Length: " + text.length();
    }
    if (value instanceof Integer number) {
        return "Integer: " + number;
    }
    return "Unsupported";
}
```

Pattern variables are available only where flow analysis proves the type test succeeded.

Switch expressions

```
static String status(int code) {
    return switch (code) {
        case 200 -> "OK";
        case 400, 422 -> "CLIENT_ERROR";
        case 500 -> "SERVER_ERROR";
        default -> "UNKNOWN";
    };
}

static int discount(int quantity) {
    return switch (quantity) {
        case 1 -> 0;
        case 2, 3 -> 10;
        default -> {
            int value = quantity > 10 ? 30 : 20;
            yield value;
        }
    };
}
```

Use yield to produce a value from a block within a switch expression. Return exits a method.

Text blocks

```
String json = """
    {
      "id": 101,
      "status": "PAID"
    }
    """;
```

Text blocks improve readability but do not escape or parameterize SQL automatically. Use PreparedStatement parameters for database inputs.

Helpful NullPointerExceptions and strong encapsulation

Java 14 introduced more informative NullPointerException messages, and Java 17 strongly encapsulates JDK internals. Code that relies on unsupported internal APIs or reflective access may need changes. Prefer supported public APIs.

Java 17 preview-feature rule

Preview features are not finalized and can change or disappear. In Java 17, pattern matching for switch was preview. If experimenting, consult the Java 17 preview documentation and required compiler/runtime flags; do not present it as a finalized Java 17 feature.

10. Date and time API

Prefer java.time over the legacy Date/Calendar APIs for most new code.

Type	Meaning
LocalDate	Calendar date without time or time zone.
LocalTime	Time of day without date or time zone.
LocalDateTime	Date and time without time zone.
Instant	Point on the UTC timeline.
ZonedDateTime	Date and time with a time zone.
Duration	Time-based amount.
Period	Date-based amount such as years/months/days.
DateTimeFormatter	Parsing and formatting date/time values.

```
import java.time.Instant;
import java.time.ZoneId;
import java.time.ZonedDateTime;
import java.time.format.DateTimeFormatter;
```

```
Instant now = Instant.now();
ZonedDateTime tokyo = now.atZone(ZoneId.of("Asia/Tokyo"));
String formatted = tokyo.format(DateTimeFormatter.ISO_ZONED_DATE_TIME);
```

Use Instant for timestamps, ZonedDateTime when local civil time and zone rules matter, and LocalDate for dates such as birthdays or accounting dates. Do not assume a day always has exactly 24 hours in a daylight-saving time zone.

11. Concurrency and the Java Memory Model

Threads and race conditions

A race condition occurs when correctness depends on the timing/interleaving of concurrent operations. The expression count++ is a read-modify-write sequence, not one atomic operation.

```
class UnsafeCounter {
    private int count = 0;
    void increment() { count++; } // not thread-safe
    int get() { return count; }
}
```

AtomicInteger

```
import java.util.concurrent.atomic.AtomicInteger;

class Counter {
    private final AtomicInteger count = new AtomicInteger();

    void increment() { count.incrementAndGet(); }
    int get() { return count.get(); }
}
```

AtomicInteger supports atomic operations for a single integer. For compound invariants across multiple variables, use a suitable lock or other coordination mechanism.

synchronized and volatile

• synchronized provides mutual exclusion and establishes visibility/order relationships through monitor lock/unlock.

• volatile provides visibility and ordering guarantees for reads/writes of a variable, but does not make compound actions such as count++ atomic.

• Use locks when critical sections require coordinated access.

• Avoid locking on publicly accessible objects or strings that may be interned.

ExecutorService and CompletableFuture

```
ExecutorService pool = Executors.newFixedThreadPool(4);
try {
    CompletableFuture<String> profile =
        CompletableFuture.supplyAsync(() -> "profile", pool);
    CompletableFuture<String> orders =
        CompletableFuture.supplyAsync(() -> "orders", pool);

    String result = profile.thenCombine(orders,
        (p, o) -> p + " + " + o).join();

    System.out.println(result);
} finally {
    pool.shutdown();
}
```

• Prefer managed executors over manually creating a new thread for every task.

• Callable returns a result and can throw checked exceptions; Runnable does not return a result.

• Future represents a pending result; CompletableFuture supports composition and callbacks.

• Handle exceptional completion, timeouts, cancellation, and executor lifecycle deliberately.

• A deadlock can occur when threads wait in a cycle for locks/resources held by one another.

Happens-before

The Java Memory Model defines when one thread's writes are guaranteed visible to another. Examples include program order within a thread, monitor unlock before a subsequent lock on the same monitor, volatile write before a subsequent read of that variable, and thread start/join relationships.

Thread-safety checklist

• Identify shared mutable state.

• Prefer immutability and confinement where possible.

• Use thread-safe collections or synchronization for shared mutation.

• Avoid blocking work on inappropriate executor pools.

• Measure throughput and latency; more threads do not automatically mean more performance.

12. JVM, memory, and garbage collection

Runtime areas

Area	Purpose
Heap	Objects and arrays; garbage-collected.
Thread stack	Per-thread frames, local execution state, and call information.
Metaspace	Class metadata in native memory.
Program counter	Per-thread execution position concept.
Native method stack	Supports native method execution.

Class lifecycle

• Loading: find or create the class representation.

• Linking: verification, preparation, and resolution as required.

• Initialization: execute class initialization logic, including static initializers and field initializers.

Garbage collection

Garbage collectors reclaim memory for objects that are no longer reachable from GC roots. They do not know whether an application still considers a reachable object useful.

• Memory leaks can occur when caches, static collections, listeners, or thread-local values retain objects unintentionally.

• OutOfMemoryError can occur when the JVM cannot allocate required resources; StackOverflowError often occurs when a thread's stack is exhausted.

• G1 is a common general-purpose collector; collector selection should follow workload measurements and operational requirements.

• Avoid calling System.gc() as a normal memory-management strategy.

• Use profilers, heap dumps, GC logs, and metrics to investigate memory problems.

JIT and warm-up

The JIT compiler can optimize frequently executed code based on runtime behavior. Microbenchmarks must account for warm-up, dead-code elimination, forks, and measurement noise; use a proper benchmarking harness such as JMH for JVM microbenchmarks.

13. I/O, NIO, modules, and useful APIs

Files and Path

```
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.List;

Path path = Path.of("notes.txt");
try {
    List<String> lines = Files.readAllLines(path);
    System.out.println(lines.size());
} catch (IOException e) {
    throw new RuntimeException("Could not read notes", e);
}
```

readAllLines loads all lines into memory; for very large files, use a streaming approach and close the stream.

Modules

The Java Platform Module System was introduced in Java 9. A module declares dependencies and packages it exports using module-info.java. Modules improve encapsulation and dependency structure but are not required for every small application.

```
module com.example.app {
    requires java.net.http;
    exports com.example.api;
}
```

Useful standard-library tools

• Objects.requireNonNull for argument validation.

• Math.addExact and multiplyExact to detect integer overflow in selected operations.

• BigDecimal for decimal arithmetic with explicit scale/rounding.

• UUID for identifiers where its properties fit the use case.

• java.net.http.HttpClient for HTTP client calls.

• java.util.concurrent for synchronization, executors, queues, and atomic types.

• java.util.regex for regular expressions; avoid catastrophic backtracking on untrusted input.

14. Clean code, testing, and production guidance

Clean code habits

• Choose names that communicate intent; keep methods focused.

• Validate at boundaries and make illegal states harder to represent.

• Prefer immutable data where practical.

• Separate domain logic from I/O and framework concerns.

• Use composition when it makes dependencies easier to change.

• Avoid overengineering and abstractions with no clear benefit.

Testing

• Unit tests verify small units of behavior; integration tests verify interactions between components.

• Test normal cases, boundaries, invalid inputs, and failure behavior.

• Prefer deterministic tests that do not depend on sleep timing or external services.

• For concurrency, test invariants and use tools/approaches designed for concurrent behavior where necessary.

• Use dependency injection or explicit collaborators to make code testable.

Production considerations

• Use logging with context and correlation identifiers, but never log passwords, secrets, or unnecessary personal data.

• Use timeouts and bounded resource pools for network calls.

• Handle retries carefully; retry only when safe and with backoff/jitter as appropriate.

• Monitor latency, error rates, saturation, and memory/GC behavior.

• Use a supported JDK distribution and patch level consistent with organizational policy.

15. DSA patterns and complexity

Big-O quick reference

Complexity	Typical example
O(1)	Array index access; average hash lookup.
O(log n)	Binary search on sorted data; balanced-tree lookup.
O(n)	Single pass over an array.
O(n log n)	Typical comparison sorting.
O(n²)	Nested loops over all pairs.
O(2^n)	Some exhaustive subset/backtracking algorithms.
O(n!)	Brute-force permutations.

Common interview patterns

Pattern	Useful when	Example prompts
Hash map/set	Fast membership, frequency, complement lookup	Two Sum; first unique element
Two pointers	Sorted arrays, pair search, in-place compaction	Valid Palindrome; 3Sum
Sliding window	Contiguous subarray/substring constraints	Longest substring without repeats
Binary search	Monotonic condition or sorted search space	Rotated sorted array; minimum feasible capacity
Stack / monotonic stack	Nested structure or next greater/smaller element	Valid Parentheses; Daily Temperatures
Heap	Repeated min/max or top-k	Top K Frequent; merge k lists
Tree DFS/BFS	Hierarchical traversal and paths	Level order; LCA
Graph traversal	Connectivity, reachability, shortest paths	Number of Islands; course schedule
Backtracking	Generate combinations/permutations under constraints	Combination Sum
Dynamic programming	Overlapping subproblems and optimal substructure	Coin Change; longest common subsequence

How to present a solution

• Clarify input size, duplicates, ordering, nulls, and expected output.

• Explain a straightforward approach first, then identify its bottleneck.

- State the invariant or recurrence before coding.
- Test normal, boundary, and adversarial cases.
- Give time and auxiliary-space complexity precisely.
- Use clear variable names and explain trade-offs.

16. Interview questions with answers

Use these as active-recall prompts: try answering aloud before reading the answer. Questions are grouped from core Java through advanced topics. Interview expectations vary by role and company.

1. Why is Java platform-independent?

Java compiles source into JVM bytecode. A compatible JVM on each target platform executes that bytecode. Native dependencies and environment-specific behavior can still limit portability.

2. What is the difference between JDK and JVM?

The JDK provides development tools such as javac and runtime components. The JVM executes bytecode. The JRE is the traditional runtime packaging concept.

3. Is Java pass-by-reference?

No. Java always passes arguments by value. For an object, the copied value is a reference to the object.

4. What is the difference between == and equals()?

For primitives, == compares values. For references, == compares identity. equals() compares logical equality when the type implements it appropriately.

5. Why override hashCode() when overriding equals()?

Hash-based collections use hash codes to find candidate buckets and equality to distinguish keys. Equal objects must have equal hash codes.

6. Overloading versus overriding?

Overloading changes the parameter list and is resolved at compile time. Overriding replaces inherited instance-method behavior and uses runtime dispatch.

7. Abstract class versus interface?

An abstract class can maintain instance state and constructors. Interfaces define contracts and can include default, static, and private methods. A class can implement multiple interfaces.

8. Is String mutable?

No. String is immutable. Operations that appear to modify it return a new String.

9. StringBuilder versus StringBuffer?

Both are mutable. StringBuilder is not synchronized and is typically preferred for local string construction; StringBuffer synchronizes its methods.

10. Checked versus unchecked exceptions?

Checked exceptions must be caught or declared. RuntimeException subclasses are unchecked. Errors represent serious conditions generally not handled as normal application flow.

11. What does try-with-resources do?

It closes AutoCloseable resources automatically when control leaves the block, including exceptional exits.

12. ArrayList versus LinkedList?

ArrayList provides O(1) indexed access and efficient append on average. LinkedList has O(n) indexed access but O(1) endpoint operations; node lookup and arbitrary-position operations still require traversal.

13. How does HashMap work at a high level?

It uses hash codes to locate buckets and equals() to distinguish keys. Collisions are handled internally; resizing and hash distribution affect performance.

14. Is HashMap thread-safe?

No. Concurrent shared mutation needs suitable coordination, such as ConcurrentHashMap for many common use cases.

15. Why should map keys be immutable?

Mutating fields involved in hashCode/equals after insertion can make a key difficult to locate in the bucket where it was placed.

16. HashSet versus TreeSet?

HashSet offers average constant-time basic operations with no sorted-order guarantee. TreeSet maintains sorted order with logarithmic basic operations.

17. What is type erasure?

Java generics are primarily implemented by erasing type parameters at runtime and inserting casts or bridge methods where needed. Generic type arguments are not generally available as runtime class information.

18. What is PECS?

Producer Extends, Consumer Super: use ? extends T when primarily reading T-like values and ? super T when inserting T values, as a guideline.

19. What is a functional interface?

An interface with exactly one abstract method. Lambdas and method references can provide implementations for compatible functional-interface targets.

20. map() versus flatMap() in streams?

map transforms each element. flatMap transforms each element to a stream-like result and flattens the results into one pipeline.

21. Why are stream intermediate operations lazy?

They build a pipeline and defer processing until a terminal operation triggers traversal, allowing fusion and short-circuiting in appropriate operations.

22. Can a stream be reused?

No. Once a terminal operation consumes a stream, it should not be reused. Create a new stream from the source.

23. orElse() versus orElseGet()?

orElse evaluates its argument eagerly. orElseGet invokes a supplier only if the Optional is empty.

24. What does var mean in Java?

It requests compile-time inference for a local variable's type from its initializer. It does not create a dynamically typed variable.

25. What is a record?

A concise data-carrier class that provides component accessors, a canonical constructor, and value-oriented equals/hashCode/toString implementations. Its immutability is shallow.

26. What is a sealed class?

A class or interface that restricts its direct permitted subtypes. Each direct subtype must declare itself final, sealed, or non-sealed.

27. Was pattern matching for switch finalized in Java 17?

No. It was a preview feature in Java 17. Records, pattern matching for instanceof, and sealed classes had already been finalized by Java 16 or 17.

28. What is the difference between yield and return?

yield provides a value from a switch-expression block. return exits the enclosing method.

29. What is the difference between trim() and strip()?

trim() uses older character rules; strip() uses Unicode-aware whitespace handling.

30. What does volatile guarantee?

It provides visibility and ordering guarantees for accesses to that variable. It does not make compound operations such as count++ atomic.

31. Why is count++ not thread-safe?

It is a read-modify-write sequence. Two threads can read the same old value and overwrite one another's increments.

32. synchronized versus volatile?

synchronized provides mutual exclusion and monitor-based happens-before guarantees. volatile provides visibility/order for a variable but no mutual exclusion.

33. What is an atomic variable?

A concurrency utility such as AtomicInteger supports atomic updates, often using compare-and-set mechanisms. It does not automatically protect multi-variable invariants.

34. What is a deadlock?

A state in which threads wait indefinitely for resources held by one another, often forming a cycle. Lock ordering, timeouts, and reducing nested locks can help prevent it.

35. Future versus CompletableFuture?

Future represents a pending result and supports operations such as get/cancel. CompletableFuture also supports composition, callbacks, and combining asynchronous stages.

36. What is the heap?

The runtime memory area where objects and arrays are generally allocated. Garbage collection reclaims unreachable objects.

37. Can Java have memory leaks?

Yes. Reachable objects can be retained unnecessarily by caches, static collections, listeners, ThreadLocal values, or other references.

38. StackOverflowError versus OutOfMemoryError?

StackOverflowError often results from excessive stack depth. OutOfMemoryError indicates failure to allocate required memory or another JVM resource.

39. What is JIT compilation?

The JVM can compile frequently executed bytecode into native machine code and optimize it using runtime observations.

40. What is happens-before?

A Java Memory Model ordering relation that guarantees visibility of one action's effects to another, for example through monitor locks, volatile access, thread start, and join.

41. Why prefer BigDecimal for money?

Binary floating point cannot represent many decimal fractions exactly. BigDecimal supports decimal arithmetic with explicit scale and rounding policy; it must still be used carefully.

42. What is the difference between LocalDateTime and Instant?

LocalDateTime has no time zone and does not identify a unique global instant. Instant represents a point on the UTC timeline.

43. Why are parallel streams not always faster?

Splitting, scheduling, merging, synchronization, ordering, and workload size can outweigh parallel gains. Benchmark representative workloads.

44. What is composition over inheritance?

Build behavior by combining collaborators instead of relying on a deep inheritance hierarchy. It often reduces coupling and makes behavior easier to change and test.

45. How should you approach a FAANG-style coding problem?

Clarify requirements, state a brute-force approach, optimize the bottleneck, explain invariants, code clearly, test edge cases, and state time/space complexity.

Additional coding prompts to practise

- Implement an LRU cache with O(1) get and put.
- Find the top K frequent elements in an integer array.
- Group a list of employees by department using streams.
- Design a thread-safe rate limiter and explain its concurrency model.
- Implement a bounded producer-consumer queue.
- Find the longest substring without repeating characters.
- Merge overlapping intervals.
- Detect a cycle in a directed graph.
- Implement an immutable value object with correct equals/hashCode.
- Design an asynchronous service that combines two independent API calls with timeout and failure handling.

17. Final revision checklist and study plan

Checklist

• I can explain JDK, JVM, bytecode, class loading, and JIT.

• I understand OOP, access modifiers, constructors, static/final, and polymorphism.

• I can explain equals/hashCode and use collections correctly.

• I can reason about HashMap collisions, resizing, and key mutability.

• I can write generic methods and explain wildcards/type erasure.

• I can use lambdas, method references, streams, collectors, and Optional.

• I can explain records, sealed types, switch expressions, and Java 17 preview status.

• I understand exceptions, try-with-resources, and java.time.

• I can distinguish atomicity, visibility, synchronization, and happens-before.

• I can use executors and CompletableFuture and reason about deadlocks.

• I understand heap, stack, metaspace, GC reachability, and common memory failures.

• I solve DSA problems with edge cases and complexity analysis.

• I can explain trade-offs and write readable, testable code.

12-week plan

Weeks	Focus	Deliverable
1–2	Core syntax, OOP, strings, exceptions	Small console app and OOP exercises.
3–4	Collections, generics, equality	Frequency counter, sorting, custom value object.
5–6	Lambdas, streams, Optional, Java 17 features	Data transformation mini-project.
7–8	JVM, memory, concurrency	Thread-safe counter and async composition exercise.
9–10	DSA patterns	Daily timed problems; review mistakes.
11–12	Low-level design and mocks	Design exercises, mock interviews, revision.

Recommended daily routine

• 20–30 minutes: revise one concept and explain it aloud.

• 30–45 minutes: write code from scratch and test edge cases.

• 30–60 minutes: solve one or two DSA problems depending on difficulty.

• 10 minutes: record mistakes, complexity, and interview takeaways.

Official references

• Java 17 language changes: https://docs.oracle.com/en/java/javase/17/language/java-language-changes-release.html

• Java 17 API documentation: https://docs.oracle.com/en/java/javase/17/docs/api/index.html

• JDK 17 release notes: https://www.oracle.com/java/technologies/javase/17all-relnotes.html

Final advice: do not optimize for memorizing every API. Optimize for understanding behavior, selecting the right abstraction, writing correct code, and explaining trade-offs clearly.